



Informe Proyecto 1

Departamento de Ingeniería de Sistemas y Computación
ISIS-1226 Diseño y Programación Orientada a Objetos

Juan David Vasquez Leyva - 202426062

David Hortua - 202510382

Juan Felipe Leon - 202510397

Universidad de los Andes

Parte de Análisis.	4
1. El problema que se resuelve	4
2. Análisis	4
3. Modelo	5
3.1 Usuario	5
3.2 Tienda de Juegos	5
3.3 Cafe	6
3.4 Factura	6
3.5 Admin e Informe	6
4. Diagrama de clases	7
5. Requerimientos Funcionales	8
Lista de excepciones	8
• Requerimientos Generales	9
• Requerimientos del Cliente	11
• Requerimientos del Empleado	13
• Requerimientos del Administrador	15
• Requerimientos de consulta (Administrador)	22
Parte de Diseño.	25
1. Introducción	25
2. Cambios al modelo de dominio	25
3. Decisiones de diseño del inventario de juegos	28
4. Decisiones de diseño de la calculadora	29
5. Decisiones de diseño de persistencia	30
6. Estructura de paquetes	30
8. Paquete de Generals	32
9. Paquete de Calculadora	33
3. Paquete de Persistencia	34
5. Paquete de Torneo	36
Parte de Diseño proyecto 2	37
7. Decisiones de diseño torneo	37
Diagramas de flujo:	38
Diagrama creación torneo amistoso:	38
Creación torneo Competitivo:	39
8. Diseño interfaces y consola	39
9. Pruebas	40
Parte de diseño Proyecto 3	41
1. Cambios al modelo de Dominio para el Proyecto 3	41

1.1 Métodos cargarDatos hechos públicos en DulcesNDados	41
1.2 Paquete interfaz como reemplazo del paquete consola	41
1.3 Autenticación diferida en el PanelEmpleado	41
2. Decisiones de Diseño de la Interfaz Gráfica Empleado	42
1. Uso de JPanel como contenedor principal	42
2. Uso de BorderLayout en PanelEmpleado	42
3. Separación entre PanelBotonesEmpleado y PanelContenidoEmpleado	42
6. Reutilización de la lógica existente	43
3. Decisiones de Diseño de la Interfaz Gráfica Admin	45
1. Uso de JPanel como contenedor principal	45
2. Uso de BorderLayout en PanelAdmin	45
3. Uso de CardLayout en el área de contenido	45
4. Barra lateral envuelta en JScrollPane	45
5. Panel reutilizable para reabastecimiento	46
6. Reutilización de los gestores del dominio	46
7. Tres visualizaciones gráficas con JFreeChart	46
4. Decisiones de Diseño de la Interfaz Gráfica Cliente	46
1. CardLayout para gestión de sesión	46
2. Uso de BorderLayout en la vista principal	47
3. CardLayout en el área de contenido del cliente	47
5. Método actualizarHeader() para retroalimentación en tiempo real	48
6. Métodos de validación reutilizables	48
7. Tablas con DefaultTableModel de solo lectura	48
8. Reutilización de los gestores del dominio	48
5. Decisiones de Diseño de Gráficas JFreeChart	49
Gráfica 1 - Diagrama de Pastel: Distribución de copias de un juego	49
Gráfica 2 - Diagrama de Barras: Ventas por periodo de 5 días	49
Gráfica 3 - Diagrama de Lineas: Evolución de reservas en una semana	49
6. Estructura de paquetes del proyecto 3	50
7. Diagrama de Clases Interfaz	51
8. Diagrama de Alto Nivel de Interfaz	51
8. Diagrama de Alto Nivel de TODO EL PROYECTO	52

Parte de Análisis.

1. El problema que se resuelve

El proyecto tiene como propósito resolver el problema de El Board Game Cafe “Dulces n Dados” el cual es un establecimiento que combina una cafetería que, a su vez, funciona como un lugar para poder jugar juegos de mesa. El negocio requiere un sistema de información que permita gestionar de manera eficiente sus operaciones, abarcando tres grandes áreas: la gestión del inventario de juegos de mesa (préstamo y venta), el funcionamiento del café (empleados, menú y facturas) y las tareas administrativas. Este documento presenta el modelo, los requerimientos funcionales y las restricciones del sistema.

2. Análisis

A partir del análisis del caso de estudio se llegó a los siguientes elementos. El sistema de “Dulces n Dados” se divide en tres grandes categorías: el servicio de préstamo de juegos y reserva de mesas, el servicio de cafetería y restaurante, y el rol del administrador del local. Además, el sistema también cuenta con tres tipos de usuario: Cliente, Empleado (Cocinero o Mesero) y Administrador, cada uno con su login y contraseña.

Con respecto al servicio de préstamo de juegos, para hacer un préstamo de un juego el cliente debe tener una mesa reservada, en la que se especifique el número de personas y si hay menores de edad, verificando que no pase la capacidad máxima del negocio. Existen tres restricciones adicionales según el tipo de juego: los juegos marcados como difíciles necesitan que haya un mesero capacitado para enseñar las reglas. Además, el café maneja un inventario de juegos a la venta, los cuales son distintos a los que están para préstamo, y tanto los clientes como los empleados pueden tenerlos, estos últimos tienen un código de descuento propio.

En cuanto a la cafetería, el menú incluye productos de pastelería y bebida, que pueden ser alcohólicas o calientes. La venta de algunos productos no está permitida dependiendo de los integrantes de la mesa. Por ejemplo, si en la mesa hay niños, entonces no se puede entregar bebidas alcohólicas. También, las bebidas calientes no pueden servirse cuando se está jugando un juego de acción. Todas las ventas tienen impuestos y además le dan punto de fidelidad al cliente y esto puede usarse como descuentos en compras.

Es importante mencionar que los empleados tienen turnos específicos que se repiten cada semana. El sistema debe garantizar que siempre haya al menos 1 cocinero y 2 meseros al mismo tiempo.

Por último, el administrador tiene acceso a la información de todos los juegos que hay en los inventarios de préstamo y de venta. Puede hacer pedidos de juegos nuevos para ambos inventarios, o reparar juegos. También, tiene un historial completo de todos los préstamos del local y su información. Además, tiene que hacer un informe con todos los datos de los juegos y café, incluyendo fechas, horas, montos, etc.

3. Modelo

El modelo se construyó identificando las entidades principales, sus atributos y las relaciones entre ellas. A continuación se presenta una descripción de cada clase y sus características. En esto nos basamos para hacer el UML.

3.1 Usuario

La clase Usuario es abstracta y tiene las siguientes subclases.

Usuario (abstract) : login : String, contraseña : String.

Cliente : idCliente : String, juegosComprados : List<JuegoDeMesa>, juegosFavoritos : List<JuegoDeMesa>, juegosReservados (maximo 2), puntosDeFidelidad : float, mesa : Mesa.

Administrador : Hereda usuario y no tiene más atributos propios.

Empleado (abstract) : turnoLaboral : List<String>, codigoDescuento : String, juegosFavoritos : List<JuegosDeMesa>. Está clase va a ser la superclase de Cocinero y Mesero.

Cocinero (hereda Empleado) : Hereda empleado y tiene mínimo un turno.

Mesero (hereda Empleado) : juegosDificiles : List<JuegoDeMesa>. Debe tener mínimo 2 turnos.

3.2 Tienda de Juegos

Gestiona el inventario de juegos para préstamo y venta.

TiendaDeJuegos : facturas : List<Factura>, inventarioJuegos : InventarioJuegos

InventarioJuegos : juegosPrestamo : List<JuegosDeMesa>, juegosVenta : List<JuegosDeMesaVenta>, historialPrestamos : List<Prestamo>

JuegoDeMesa : idJuego : String, enUso : boolean, añoPublicacion : datetime, empresaProduccion : String, tipoDeJuego : String, minJugadores : int, maxJugadores : int, edadMinima : int, categoria (Cartas / Tablero / Acción) : String, estado : String, dificil : boolean.

Prestamo : idJuego : String, idMesa : String, fechaInicio : datetime, fechaFin : datetime, cliente : String.

3.3 Cafe

Gestiona el funcionamiento de la cafetería.

Cafe : mesa : List<Mesa>, capacidadMaxClientes : int, empleados : List<Empleado>, menuProductos : List<Producto>, facturas : List <Factura>

Mesa : idMesa : String, cantidadClientes : int, tieneNinos : boolean, tieneJovenes : boolean, cliente : Cliente.

Turno : diaSemana : datetime, empleado : Empleado.

Producto (abstract) : nombre : String, precio : float. Esta clase es la superclase de Bebida y Pasteleria.

Bebida (hereda Producto) : alcoholica : boolean, caliente : boolean.

Pasteleria (hereda Producto) : alergenos : List<String>

3.4 Factura

Gestiona los registros de las ventas.

Factura : fecha : datetime, subtotal : float, impuestos : float, propina : float, detalles : List<DetalleVenta>

DetalleVenta : producto : Producto, cantidad : int, subtotal : float.

3.5 Admin e Informe

La clase Administrador debe hacer un informe que debe contener lo siguiente.

Informe : totalJuegos : float, totalPlatillos : float, subTotalJuegos : float, subTotalPlatillos : float, totalImpuestos : float, totalPropinas : float, totalCosto : float, fechaInicio : datetime, fechaFin : datetime.

```

classDiagram
    class Usuario {
        <<abstract>>
        +login: String
        +contraseña: String
    }
    class Cliente {
        +idCliente: String
        +juegosComprados: List<JuegoDeMesa>
        +juegosFavoritos: List<JuegoDeMesa>
        +juegosReservados: JuegoDeMesa[]*2
        +puntosDeFidelidad: float
        +mesa: Mesa
    }
    class Administrador {
    }
    class Empleado {
        <<abstract>>
        +turnoLaboral: List<String>
        +codigoDescuento: String
        +juegosFavoritos: List<JuegoDeMesa>
    }
    class Cocinero {
    }
    class Mesero {
        +juegosDificiles: List<JuegoDeMesa>
    }
    class Turno {
        +diaSemana: datetime
        +empleado: Empleado
    }
    class Mesa {
        +idMesa: String
        +cantidadClientes: int
        +tieneNinos: boolean
        +tieneJovenes: boolean
        +cliente: Cliente
    }
    class TiendaDeJuegos {
        +facturas: List<Factura>
        +inventarioJuegos: InventarioJuegos
    }
    class InventarioJuegos {
        +juegosPrestamo: List<JuegoDeMesa>
        +juegoVenta: List<JuegoDeMesaVenta>
        +historialPrestamos: List<Prestamo>
    }
    class Prestamo {
        +idJuego: String
        +idMesa: String
        +fechaInicio: datetime
        +fechaFin: datetime
        +cliente: String
    }
    class Factura {
        +fecha: datetime
        +subtotal: float
        +impuestos: float
        +propina: float
        +detalles: List<DetalleVenta>
    }
    class DetalleVenta {
        +producto: Producto
        +cantidad: int
        +subtotal: float
    }
    class Cafe {
        +mesa: List<Mesa>
        +capacidadMaxClientes: int
        +empleados: List<Empleado>
        +menuProductos: List<Producto>
        +facturas: List<Factura>
    }
    class Producto {
        <<abstract>>
        +nombre: String
        +precio: float
    }
    class Bebida {
        +alcoholica: boolean
        +caliente: boolean
    }
    class Pasteleria {
        +alergenos: List<String>
    }
    class JuegoDeMesa {
        +idJuego: String
        +enUso: boolean
        +añoPublicacion: datetime
        +empresaProduccion: String
        +tipoDeJuego: String
        +minJugadores: int
        +maxJugadores: int
        +edadMinima: int
        +categoria: String
        +estado: String
        +dificil: boolean
    }
    Usuario <|-- Cliente
    Usuario <|-- Administrador
    Empleado <|-- Cocinero
    Empleado <|-- Mesero
    Empleado <|-- Turno
    Cliente --> Empleado : Extends
    Empleado --> Turno : Extends
    Empleado --> Cocinero : Extends
    Empleado --> Mesero : Extends
    Cliente --> Mesa : mesa
    Mesa --> Cliente : cliente
    TiendaDeJuegos --> Factura : facturas
    Factura --> TiendaDeJuegos : facturas
    Factura --> DetalleVenta : detalles
    DetalleVenta --> Factura : detalles
    Cafe --> Empleado : empleados
    Cafe --> Producto : menuProductos
    Cafe --> Factura : facturas
    Producto <|-- Bebida
    Producto <|-- Pasteleria
    Producto --> JuegoDeMesa : juegosFavoritos
    JuegoDeMesa --> Producto : juegosFavoritos
    InventarioJuegos --> Prestamo : historialPrestamos
    Prestamo --> InventarioJuegos : historialPrestamos
    
```

The diagram illustrates a bar management system with the following components:

- Usuario (abstract)**: Base class for users, with attributes `login: String` and `contraseña: String`.
- Cliente**: Inherits from `Usuario`. Attributes include `idCliente: String`, `juegosComprados: List<JuegoDeMesa>`, `juegosFavoritos: List<JuegoDeMesa>`, `juegosReservados: JuegoDeMesa[]*2`, `puntosDeFidelidad: float`, and `mesa: Mesa`.
- Administrador**: Inherits from `Usuario`. Associated with `Informe`.
- Empleado (abstract)**: Base class for staff, with attributes `turnoLaboral: List<String>`, `codigoDescuento: String`, and `juegosFavoritos: List<JuegoDeMesa>`.
- Cocinero**: Inherits from `Empleado`.
- Mesero**: Inherits from `Empleado`. Attribute: `juegosDificiles: List<JuegoDeMesa>`.
- Turno**: Inherits from `Empleado`. Attributes: `diaSemana: datetime` and `empleado: Empleado`.
- Mesa**: Associated with `Cliente`. Attributes: `idMesa: String`, `cantidadClientes: int`, `tieneNinos: boolean`, `tieneJovenes: boolean`, and `cliente: Cliente`.
- TiendaDeJuegos**: Associated with `Factura`. Attributes: `facturas: List<Factura>` and `inventarioJuegos: InventarioJuegos`.
- InventarioJuegos**: Associated with `Prestamo`. Attributes: `juegosPrestamo: List<JuegoDeMesa>`, `juegoVenta: List<JuegoDeMesaVenta>`, and `historialPrestamos: List<Prestamo>`.
- Prestamo**: Associated with `InventarioJuegos`. Attributes: `idJuego: String`, `idMesa: String`, `fechaInicio: datetime`, `fechaFin: datetime`, and `cliente: String`.
- Factura**: Associated with `TiendaDeJuegos` and `DetalleVenta`. Attributes: `fecha: datetime`, `subtotal: float`, `impuestos: float`, `propina: float`, and `detalles: List<DetalleVenta>`.
- DetalleVenta**: Associated with `Factura`. Attributes: `producto: Producto`, `cantidad: int`, and `subtotal: float`.
- Cafe**: Associated with `Empleado`, `Producto`, and `Factura`. Attributes: `mesa: List<Mesa>`, `capacidadMaxClientes: int`, `empleados: List<Empleado>`, `menuProductos: List<Producto>`, and `facturas: List<Factura>`.
- Producto (abstract)**: Base class for products, with attributes `nombre: String` and `precio: float`.
- Bebida**: Inherits from `Producto`. Attributes: `alcoholica: boolean` and `caliente: boolean`.
- Pasteleria**: Inherits from `Producto`. Attribute: `alergenos: List<String>`.
- JuegoDeMesa**: Associated with `Producto`. Attributes: `idJuego: String`, `enUso: boolean`, `añoPublicacion: datetime`, `empresaProduccion: String`, `tipoDeJuego: String`, `minJugadores: int`, `maxJugadores: int`, `edadMinima: int`, `categoria: String`, `estado: String`, and `dificil: boolean`.

Relationships include inheritance (indicated by hollow triangle arrows), associations (solid lines with role names), and many-to-many associations (indicated by crow's foot notation).

La clase Empleado es abstracta y tiene como subclases a Mesero y Cocinero. El mesero debe conocer juegos difíciles, mientras que el cocinero es un empleado común y corriente. Por su parte, el Administrador tiene un historial completo de los préstamos realizados en el local, junto con su información, además, debe hacer informes detallados de las ventas.

El cliente se relaciona con la mesa a través de la reserva y con los juegos de mesas por medio de la compra o pedirlos prestados. Además, al tener una mesa, puede hacer uso de las funciones de la cafetería, como ordenar productos tanto de pastelería como bebidas. Es importante mencionar que, tanto las bebidas, los productos de pastelería y los juegos de mesa son herederos de la clase Producto. Cada vez que se hace una compra, al cliente se le genera una factura por cada compra de juegos y cada gasto en el café.

Finalmente, es importante mencionar que los inventarios de préstamo y de venta se separaron dentro de la clase `InventarioJuegos` para mantener la distinción entre ambos, ya que el texto menciona que ambas tienen listas completamente diferentes.

5. Requerimientos Funcionales

A continuación se presentan los requerimientos funcionales del sistema, organizados por el actor principal que los ejecuta. Para cada requerimiento se describe su propósito, las entradas que recibe, la salida que produce y las excepciones que puede lanzar.

Lista de excepciones

El sistema utiliza las siguientes excepciones personalizadas para señalar condiciones de error de forma precisa:

- **EntidadNoEncontradaException** : Se lanza cuando un objeto buscado por ID no existe en el sistema.
- **CapacidadExcedidaException** : Se lanza cuando se intenta agregar más personas de las que permite la capacidad del café.
- **JuegoNoDisponibleException** : Se lanza cuando se intenta prestar un juego que ya está en uso.
- **RestriccionEdadException** : Se lanza cuando el juego solicitado no es apto para la edad de los comensales de la mesa.
- **RestriccionJugadoresException** : Se lanza cuando el número de personas en la mesa está fuera del rango de jugadores del juego.
- **ConflictoHorarioException** : Se lanza cuando la asignación o cambio de turno viola el mínimo de empleados requerido.
- **OperacionNoPermitidaException** : Se lanza cuando un actor intenta realizar una acción para la que no tiene permisos.
- **InventarioInsuficienteException** : Se lanza cuando no hay suficientes unidades de un juego en el inventario.
- **SolicitudNoValidaException** : Se lanza cuando una solicitud de cambio de turno o de informe tiene parámetros inválidos.

- **EstadoInvalidoException** : Se lanza cuando se intenta realizar una operación sobre un objeto que no está en el estado adecuado.
- **CupoNoDisponible**: Se lanza cuando se hacer una inscripción invalida por los cupos disponibles para cuposNormales o cuposJuegoFavorito

- **Requerimientos Generales**

1. Iniciar sesión

Actor

Administrador, Cliente y Empleado

Descripción

Los usuarios del sistema podrán autenticarse utilizando su login y contraseña para acceder a las funcionalidades correspondientes a su rol dentro de la aplicación.

Entradas

- Login del usuario
- Contraseña del usuario

Salidas

- Acceso exitoso al sistema
- Mensaje de error en caso de credenciales inválidas

12. Inscripción a torneos

Actor

Cliente y Empleado

Descripción

Los usuarios podrán inscribirse a torneos disponibles en el sistema. Cada inscripción podrá contener un máximo de tres participantes y deberá asociarse a un torneo específico.

En torneos competitivos, los empleados podrán participar sin pagar tarifa de entrada.

Entradas

- Torneo seleccionado
- Lista de participantes

Salidas

- Confirmación de inscripción exitosa
- Mensaje de error si:
 - no existen cupos disponibles
 - la inscripción supera el máximo permitido
 - el torneo no existe

13. Desinscripción de torneos

Actor

Cliente y Empleado

Descripción

Los usuarios podrán desinscribirse de un torneo previamente registrado. Al realizar la desinscripción, el sistema eliminará completamente la inscripción asociada al usuario.

Entradas

- Usuario autenticado
- Torneo seleccionado

Salidas

- Confirmación de desinscripción exitosa
- Liberación de los cupos correspondientes
- Mensaje de error si el usuario no se encuentra inscrito

● Requerimientos del Cliente

reservarJuego

Actor: Cliente

Descripción: Permite a un cliente solicitar el préstamo de un juego de mesa para su mesa. El sistema verifica que el juego esté disponible, que sea apto para el número de personas y las edades presentes en la mesa, y que la capacidad del café no se haya superado. Retorna un objeto Prestamo que registra el inicio del préstamo.

Entradas:

- **clientId:** String
- **juegoId:** String
- **mesaId:** String

Salida:

- Prestamo

Excepciones:

- JuegoNoDisponibleException
- RestriccionEdadException
- RestriccionJugadoresException
- CapacidadExcedidaException

solicitarJuegoPrestado

Actor: Cliente

Descripción: Variante de reservarJuego que también permite a un cliente solicitar un juego prestado para su mesa. Al igual que reservarJuego, valida disponibilidad, restricciones de edad y de número de jugadores antes de registrar el préstamo.

Entradas:

- **clientId:** String
- **mesaId:** String
- **juegoId:** String

Salida:

- Prestamo

Excepciones:

- JuegoNoDisponibleException
- RestriccionEdadException
- RestriccionJugadoresException

comprarJuegoCliente

Actor: Cliente

Descripción: Permite a un cliente comprar uno o varios juegos del inventario de venta en una sola transacción. El sistema verifica que haya suficientes unidades disponibles, aplica el IVA del 19% y genera una factura detallada. Si el cliente tiene un código de descuento válido, este se aplica antes del cálculo de impuestos.

Entradas:

- **clienteId:** String
- **listaJuegos:** List<JuegoDeMesa>

Salida:

- Factura

Excepciones:

- InventarioInsuficienteException

agregarPuntosDeFidelidad hecho

Actor: Cliente (automático tras cada compra)

Descripción: Calcula y acumula los puntos de fidelidad del cliente tras cada compra. El sistema agrega el 1% del monto de la compra como puntos, donde cada punto equivale a un peso y puede ser usado como descuento en compras futuras.

Entradas:

- **clienteId:** String
- **montoCompra:** float

Salida:

- float (puntos agregados)

Excepciones:

- EntidadNoEncontradaException

2. Registro de clientes

Actor

Cliente

Descripción

Un cliente podrá registrarse de manera autónoma en el sistema proporcionando su información personal para obtener acceso a las funcionalidades de la aplicación.

Entradas

- Nombre
- Login
- Contraseña
- Información personal requerida

Salidas

- Confirmación de registro exitoso
- Mensaje de error si el usuario ya existe o los datos son inválidos

• Requerimientos del Empleado

solicitarCambioHorario hecho

Actor: Empleado

Descripción: Permite a un empleado solicitar el cambio de uno de sus turnos semanales a otro día disponible. El sistema verifica que el cambio no deje al café con menos del mínimo requerido de cocineros (1) o meseros (2) en ningún turno afectado. La solicitud queda en estado pendiente hasta que el administrador la apruebe o rechace.

Entradas:

- **empleadoid:** String
- **turnoActual:** Turno

- **turnoNuevo:** Turno

Salida:

- SolicitudCambio

Excepciones:

- SolicitudNoValidaException

solicitarIntercambioHorario hecho

Actor: Empleado

Descripción: Permite a dos empleados intercambiar sus turnos entre sí. El sistema verifica que el intercambio no viole los mínimos de cobertura del café en ninguno de los días involucrados. La solicitud queda pendiente hasta la aprobación del administrador.

Entradas:

- **empleado1Id:** String
- **empleado2Id:** String
- **turno1:** Turno
- **turno2:** Turno

Salida:

- SolicitudCambio

Excepciones:

- SolicitudNoValidaException

sugerirPlatillo hecho

Actor: Empleado

Descripción: Permite a cualquier empleado (cocinero o mesero) proponer un nuevo producto para el menú del café. La sugerencia queda registrada en el sistema a la espera de que el administrador la evalúe y decida si la incorpora al menú.

Entradas:

- **empleadId:** String
- **producto:** Producto

Salida:

- void

Excepciones:

- OperacionNoPermitidaException

- **Requerimientos del Administrador**

repararJuego

Actor: Administrador

Descripción: Permite al administrador registrar la reparación de un juego del inventario de préstamo. El estado del juego debe ser distinto de 'Nuevo' para poder repararse. Internamente, se reemplaza la copia dañada por una del inventario de ventas y se actualiza el estado del juego.

Entradas:

- **juegold:** String

Salida:

- void

Excepciones:

- EstadoInvalidoException
- EntidadNoEncontradaException

darJuegoPorRobado

Actor: Administrador

Descripción: Permite al administrador marcar un juego como desaparecido cuando no ha sido devuelto tras un período razonable. El juego se elimina del inventario activo y su estado se actualiza a 'Robado', quedando registrado en el historial de préstamos.

Entradas:

- **juegold:** String

Salida:

- void

Excepciones:

- EntidadNoEncontradaException

venderJuego

Actor: Administrador

Descripción: Permite al administrador registrar la venta directa de un juego a un cliente desde el inventario de venta. El sistema verifica que haya stock disponible, aplica los impuestos correspondientes y genera una factura.

Entradas:

- **clienteId:** String
- **juegoId:** String
- **cantidad:** int

Salida:

- Factura

Excepciones:

- InventarioInsuficienteException
- EntidadNoEncontradaException

asignarHorario

Actor: Administrador

Descripción: Permite al administrador asignar un nuevo turno a un empleado. El sistema verifica que la asignación no genere conflictos de cobertura mínima del café antes de confirmarla.

Entradas:

- **empleadId:** String
- **turno:** Turno

Salida:

- void

Excepciones:

- ConflictoHorarioException

asignarTurnoHorario

Actor: Administrador

Descripción: Variante de asignarHorario que permite al administrador establecer el turno semanal de un empleado específico. Valida las mismas restricciones de cobertura mínima.

Entradas:

- **empleadoid:** String
- **turno:** Turno

Salida:

- void

Excepciones:

- ConflictoHorarioException

modificarTurnoHorario

Actor: Administrador

Descripción: Permite al administrador modificar el turno existente de un empleado, reemplazando el turno antiguo por uno nuevo. El sistema valida que el cambio no deje al café sin la cobertura mínima requerida en ninguno de los días involucrados.

Entradas:

- **empleadoid:** String
- **turnoAntiguo:** Turno
- **turnoNuevo:** Turno

Salida:

- void

Excepciones:

- ConflictoHorarioException

aprobarSolicitudHorario

Actor: Administrador

Descripción: Permite al administrador revisar y aprobar una solicitud de cambio o intercambio de turno previamente enviada por un empleado. Una vez aprobada, el sistema actualiza los turnos correspondientes y notifica a los empleados involucrados.

Entradas:

- **administradorId:** String
- **solicitudId:** String

Salida:

- void

Excepciones:

- EntidadNoEncontradaException

aceptarSugerenciaPlatillo

Actor: Administrador

Descripción: Permite al administrador revisar una sugerencia de platillo enviada por un empleado y aprobarla para que sea incorporada al menú del café.

Entradas:

- **administradorId:** String
- **producto:** Producto

Salida:

- void

Excepciones:

- EntidadNoEncontradaException

agregarPlatillo

Actor: Administrador

Descripción: Permite al administrador agregar directamente un nuevo producto al menú del café, sin necesidad de que provenga de una sugerencia. El producto debe tener un estado válido para poder ser incorporado.

Entradas:

- **producto:** Producto

Salida:

- void

Excepciones:

- EstadoInvalidoException

moverJuegoDeVentasAPrestamo

Actor: Administrador

Descripción: Permite al administrador mover un juego del inventario de ventas al inventario de préstamos. El sistema verifica que haya unidades suficientes en el inventario de ventas antes de realizar el movimiento.

Entradas:

- **juegold:** String

Salida:

- void

Excepciones:

- InventarioInsuficienteException

reabastecerInventarioVentas

Actor: Administrador

Descripción: Permite al administrador comprar y agregar nuevas unidades de un juego al inventario de ventas. El juego debe estar en un estado válido para poder ser reabastecido.

Entradas:

- **juegold:** String
- **cantidad:** int

Salida:

- void

Excepciones:

- EstadoInvalidoException

reabastecerInventarioPrestamo

Actor: Administrador

Descripción: Permite al administrador comprar y agregar nuevas unidades de un juego al inventario de préstamo. Al igual que con el inventario de ventas, el juego debe estar en un estado válido.

Entradas:

- **juegold:** String
- **cantidad:** int

Salida:

- void

Excepciones:

- EstadoInvalidoException

reportarJuegoRobado

Actor: Administrador

Descripción: Permite al administrador reportar formalmente un juego como robado. El juego se marca como desaparecido en su estado y queda registrado en el historial de préstamos para efectos contables e históricos.

Entradas:

- **juegold:** String

Salida:

- void

Excepciones:

- EntidadNoEncontradaException

3. Registro de empleados

Actor

Administrador

Descripción

El administrador podrá registrar empleados dentro del sistema para permitirles acceder a las funcionalidades correspondientes a su rol.

Entradas

- Nombre del empleado
- Login
- Contraseña
- Información laboral requerida

Salidas

- Confirmación de registro exitoso
- Mensaje de error si el empleado ya existe o los datos son inválidos

11. Creación de torneos

Actor

Administrador

Descripción

El administrador podrá crear torneos amistosos o competitivos asociados a un juego de mesa y a un día específico de la semana. Cada torneo tendrá un número máximo de cupos, reservando el 20% para usuarios cuyo juego favorito coincida con el juego del torneo.

Entradas

- Tipo de torneo
- Juego de mesa asociado
- Día del torneo
- Cantidad máxima de participantes

Salidas

- Confirmación de creación del torneo

- Mensaje de error si la información ingresada es inválida

14. Asignación de ganador de torneo

Actor

Administrador

Descripción

El administrador podrá asignar el ganador de un torneo finalizado. Si el torneo es competitivo y el ganador no es un empleado, el sistema otorgará como premio la totalidad del dinero recaudado en puntos de fidelidad. Si el torneo es amistoso y el ganador no es un empleado, el sistema otorgará un bono porcentual de descuento.

Entradas

- Torneo finalizado
- Usuario ganador

Salidas

- Registro del ganador
- Asignación del premio correspondiente
- Mensaje de error si el usuario no puede recibir premios

● Requerimientos de consulta (Administrador)

consultarEstadoJuego

Actor: Administrador

Descripción: Permite al administrador consultar el estado actual de un juego específico del inventario (Nuevo, Bueno, Falta una pieza, Robado, etc.). Esta información es exclusiva del administrador y no es visible para los clientes.

Entradas:

- **juegold:** String

Salida:

- String (estado actual del juego)

Excepciones:

- EntidadNoEncontradaException

consultarNumPrestamosJuego

Actor: Administrador

Descripción: Permite al administrador consultar cuántas veces ha sido prestado un juego específico a lo largo de toda la historia del café. Es útil para identificar los juegos más populares y tomar decisiones de inventario.

Entradas:

- **juegold:** String

Salida:

- int (número total de préstamos)

Excepciones:

- EntidadNoEncontradaException

consultarFechaPrestamoJuego

Actor: Administrador

Descripción: Permite al administrador consultar todas las fechas en las que un juego específico ha sido prestado. Retorna una lista de fechas para tener el historial completo de uso del juego.

Entradas:

- **juegold:** String

Salida:

- List<datetime> (fechas de préstamo)

Excepciones:

- EntidadNoEncontradaException

generarInforme

Actor: Administrador

Descripción: Permite al administrador generar un informe financiero detallado del café para un período de tiempo determinado. El informe incluye el total de ventas de juegos y de cafetería, separados por subtotal, impuestos y propinas, con granularidad diaria, semanal o mensual según el rango de fechas indicado.

Entradas:

- **fechaInicio:** datetime
- **fechaFin:** datetime

Salida:

- Informe

Excepciones:

- SolicitudNoValidaException

Parte de Diseño.

1. Introducción

Este documento registra todas las decisiones de diseño tomadas durante la implementación del sistema Dulces & Dados, comparando el modelo de análisis original (Entrega 1) con el código y UML actuales. Para cada decisión se indica que cambio respecto al análisis inicial y por qué se tomó esa decisión.

2. Cambios al modelo de dominio

Durante la implementación se identificaron varias diferencias respecto al modelo planteado en la Entrega 1. A continuación se muestra el nuevo modelo en UML, y se documenta cada cambio con su justificación.

D1. JuegoDeMesa se separo en JuegoDeMesa y JuegoDeMesaFisico

Cambio: El analisis original tenia una sola clase JuegoDeMesa con atributos como estado y enUso. En el codigo actual existen dos clases: JuegoDeMesa (datos del juego como concepto: nombre, precio, minJugadores, dificil) y JuegoDeMesaFisico (extiende JuegoDeMesa y agrega estadoDelJuego y prestado) que representa una copia fisica especifica.

Justificacion: El enunciado distingue entre la informacion del juego visible para clientes y el estado de cada copia fisica visible solo para el administrador. Una sola clase mezclaba ambas responsabilidades. Con la separacion, JuegoDeMesa es el catalogo publico y JuegoDeMesaFisico es la copia real que puede estar prestada, danada o robada.

D2. JuegoDeMesa extiende Producto del paquete cafe

Cambio: En el analisis, JuegoDeMesa y Producto eran entidades separadas sin relacion de herencia. En el codigo actual, JuegoDeMesa extiende Producto, compartiendo nombre y precio.

Justificacion: Esto permite que DetalleVenta pueda referenciar cualquier tipo de producto (bebida, pasteleria o juego de mesa) con un solo tipo, unificando el sistema de facturacion para todos los tipos de venta sin necesitar clases de detalle separadas.

D3. categoria de JuegoDeMesa reemplazada por TipoDeJuego como enum

Cambio: El analisis tenia categoria : String con valores Cartas, Tablero o Accion. En el codigo existe un enum TipoDeJuego con esos mismos valores como constantes tipadas.

Justificacion: Usar un enum evita errores por erratas al escribir el nombre de la categoria como String (por ejemplo, escribir 'accion' en lugar de 'Accion'). El compilador detecta valores invalidos en tiempo de compilacion, haciendo el codigo mas robusto.

D4. estado de JuegoDeMesaFisico quedo en JuegoDeMesaFisico, no en JuegoDeMesa

Cambio: El analisis tenia estado : String en JuegoDeMesa. En el codigo actual, estado se llama estadoDelJuego y vive en JuegoDeMesaFisico.

Justificacion: El estado (Nuevo, Bueno, Falta una pieza, Robado) describe una copia fisica especifica, no el juego en abstracto. Moverlo a JuegoDeMesaFisico es mas correcto conceptualmente: el juego Catan como concepto no tiene estado, pero la copia numero 3 del Catan si puede estar danada.

D5. Cliente usa JuegoDeMesaFisico en lugar de JuegoDeMesa

Cambio: El analisis tenia juegosComprados : List<JuegoDeMesa> y juegosReservados en Cliente. En el codigo actual, Cliente tiene juegosComprados : ArrayList<JuegoDeMesaFisico> y juegosPrestados : ArrayList<JuegoDeMesaFisico>.

Justificacion: Un cliente no compra o toma prestado el concepto abstracto de un juego, sino una copia fisica especifica. Usar JuegoDeMesaFisico permite saber exactamente que copia tiene el cliente y cual es su estado, lo cual es necesario para el historial del administrador.

D6. Mesa agrego productosOrdenados y tieneBebidaCaliente

Cambio: El analisis no incluia estos atributos en Mesa. En el codigo actual Mesa tiene productosOrdenados : ArrayList<Producto> que registra los productos pedidos durante la visita, y tieneBebidaCaliente : boolean para validar la restriccion con juegos de Accion.

Justificacion: La mesa necesita saber que productos se han ordenado para que GestorVentaCafe pueda calcular el subtotal de la factura directamente desde la mesa. El flag tieneBebidaCaliente es necesario para la restriccion del enunciado: no se puede prestar un juego de Accion si hay bebida caliente en la mesa, y viceversa.

D7. Cafe renombro facturas por catalogoPlatillos y agrego agregarMesa

Cambio: El analisis tenia facturas : List<Factura> en Cafe. En el codigo actual Cafe tiene catalogoPlatillos : ArrayList<Producto> para el menu, y las facturas se movieron a DataBase. Ademas, Cafe tiene el metodo agregarMesa() que crea y agrega mesas internamente.

Justificacion: El catalogo de platillos y las facturas son responsabilidades distintas. Las facturas son datos contables compartidos con la tienda de juegos, por lo que viven en DataBase. El

metodo agregarMesa en Cafe encapsula la creacion de mesas: nadie de afuera modifica la lista directamente, lo que respeta el principio de encapsulamiento.

D8. Turno cambio de tener un solo Empleado a tener listas de Meseros y Cocineros

Cambio: El analisis tenia Turno con un solo atributo empleado : Empleado. En el codigo actual Turno tiene meseros : ArrayList<Mesero> y cocineros : ArrayList<Cocinero>, y su constructor valida que haya minimo 2 meseros y 1 cocinero al crearlo.

Justificacion: El enunciado dice que siempre debe haber minimo 1 cocinero y 2 meseros simultaneamente. Un turno con un solo empleado no podia representar esa restriccion. Con listas separadas por tipo, el turno puede validar los minimos en el constructor y en los metodos agregarMesero, eliminarMesero, agregarCocinero y eliminarCocinero, garantizando la restriccion en todo momento.

D9. DiaSemana se implemento como enum

Cambio: El analisis tenia diaSemana : datetime en Turno. En el codigo actual existe un enum DiaSemana con los valores LUNES, MARTES, MIERCOLES, JUEVES, VIERNES, SABADO, DOMINGO.

Justificacion: Un turno es recurrente semanal, no una fecha especifica. Usar datetime implicaria inventar una fecha arbitraria para algo que conceptualmente no tiene fecha. El enum representa exactamente el dia de la semana sin fecha, y DataBase usa un array de 7 posiciones (una por dia) indexado por este enum.

D10. DataBase centraliza facturas, usuarios, turnos, solicitudes e informes

Cambio: El analisis no tenia una clase DataBase. Las facturas vivian en Cafe y en TiendaDeJuegos por separado. En el codigo actual existe DataBase con facturas, usuarios, turnos (Turno[7]), solicitudes e informes, compartida entre los modulos.

Justificacion: Las facturas, usuarios y turnos son datos compartidos entre el modulo de cafe y el de tienda de juegos. Tenerlos separados en cada modulo generaria duplicacion e inconsistencias. DataBase es el contenedor unico de los datos persistentes del sistema que trascienden un solo modulo.

D11. Turnos en DataBase como array de 7 posiciones

Cambio: El analisis no especificaba como se almacenaban los turnos. En el codigo actual DataBase tiene turnos : Turno[7], un array de exactamente 7 posiciones, una por cada dia de la semana.

Justificacion: Los dias de la semana son exactamente 7 y no cambian. Un array de tamano fijo es mas eficiente que una lista para este caso porque el acceso por indice es $O(1)$ y no hay

necesidad de buscar por nombre de día. Cada posición del array corresponde a un día según DíaSemana.

D12. Solicitud como clase abstracta con jerarquía de subclases

Cambio: El análisis mencionaba solicitudes de cambio e intercambio de turno y sugerencias de platillos sin modelarlas como clases. En el código existen: Solicitud (abstracta), SolicitudTurno, SolicitudCambioTurno (extiende SolicitudTurno), SolicitudIntercambioTurno y SolicitudSugerenciaPlatillo.

Justificación: El enunciado describe tres flujos de aprobación distintos pero con el mismo ciclo de vida: solicitar, quedar pendiente, aprobar o rechazar. Modelarlos como subclases de una clase común permite tratarlos de forma uniforme en DataBase y en el flujo de aprobación del administrador. Además, Empleado tiene los métodos solicitarCambioHorario y solicitarIntercambioHorario que crean las solicitudes directamente.

3. Decisiones de diseño del inventario de juegos

D13. InventarioJuegos usa HashMap<String, ArrayList<JuegoDeMesaFisico>>

Cambio: El análisis tenía juegosPrestamo : List<JuegoDeMesa> y juegosVenta : List<JuegoDeMesa> como listas planas. En el código actual InventarioJuegos tiene inventarioPrestamo y inventarioVenta como HashMap<String, ArrayList<JuegoDeMesaFisico>>, donde la llave es el nombre del juego y el valor es la lista de copias físicas.

Justificación: Con una lista plana no hay forma eficiente de saber cuántas copias hay de un juego específico. Con el HashMap, obtener todas las copias de Catan es simplemente inventarioPrestamo.get('Catan'), y contar las disponibles es .size() en O(1). Además, agregar nuevas copias con computeIfAbsent evita verificar si la llave existe antes de agregar.

D14. InventarioJuegos agrego catalogoJuegos como HashMap<String, JuegoDeMesa>

Cambio: El análisis no tenía un catálogo separado. En el código actual InventarioJuegos tiene catalogoJuegos : HashMap<String, JuegoDeMesa> que mapea el nombre del juego a su información conceptual.

Justificación: Los inventarios de préstamo y venta contienen copias físicas (JuegoDeMesaFisico), pero para mostrar el catálogo al cliente se necesita la información conceptual del juego (nombre, descripción, reglas) sin exponer el estado de cada copia. El catálogo permite separar la vista del cliente de los detalles del inventario.

D15. TipoInventario como enum para diferenciar operaciones

Cambio: El analisis no tenia este concepto. En el codigo actual existe el enum TipoInventario con valores VENTAS y PRESTAMO, usado como parametro en validarDisponibilidadJuegos, eliminarJuegoInventario, eliminarPrimerJuego y agregarJuegoInventario.

Justificacion: Sin este enum, habria que duplicar cada metodo del inventario o usar un String como parametro. El enum garantiza que solo se puedan usar los valores validos, detecta errores en compilacion y hace el codigo mas legible. Un metodo como validarDisponibilidadJuegos(nombre, cantidad, TipoInventario.VENTAS) es autoexplicativo.

D16. InventarioJuegos valida disponibilidad antes de eliminar

Cambio: El analisis no detallaba este flujo. En el codigo actual, antes de cualquier eliminacion del inventario se llama a validarDisponibilidadJuegos que lanza JuegoNoDisponibleException si la cantidad solicitada supera el stock. Luego eliminarJuegoInventario hace la eliminacion buscando por idJuego especifico, y eliminarPrimerJuego quita la primera copia disponible.

Justificacion: Separar la validacion de la eliminacion respeta el principio de responsabilidad unica. La validacion protege el inventario de operaciones invalidas antes de modificarlo. Tener dos metodos de eliminacion (por ID especifico vs primer disponible) permite manejar los casos de reparacion de un juego especifico y prestamo del primero disponible de forma distinta.

4. Decisiones de diseño de la calculadora

D17. Calculadora como interfaz con constantes y metodos default

Cambio: El analisis no tenia esta clase. En el codigo actual existe la interfaz Calculadora con las constantes IVA=0.19, impuestoConsumoAsociado=0.08, descuentoEmpleados=0.2, descuentoClientes=0.1 y propinaBase=0.1. Tiene metodos default para calcularDescuento, calcularPropina, calcularPuntosFidelidad y calcularSubtotal, y un metodo abstracto calcularImpuestos.

Justificacion: Centralizar las operaciones matematicas en una interfaz evita duplicar logica entre GestorVentaJuegos y GestorVentaCafe. Los metodos default permiten que las implementaciones hereden el comportamiento comun sin repetir codigo. Solo calcularImpuestos es abstracto porque es la unica operacion que difiere entre los dos tipos de venta.

D18. CalculadoraVentaJuegos y CalculadoraVentaCafe como implementaciones separadas

Cambio: El analisis no tenia estas clases. En el codigo actual CalculadoraVentaJuegos implementa calcularImpuestos con IVA del 19%, y CalculadoraVentaCafe implementa calcularImpuestos con impuesto al consumo del 8%. CalculadoraVentaCafe agrega ademas calcularTotal(Mesa mesa, double propina).

Justificacion: El enunciado establece impuestos distintos por tipo de venta. Dos clases concretas permiten que cada gestor instancie su propia calculadora sin saber que formula usa el otro. El metodo calcularTotal en CalculadoraVentaCafe encapsula el calculo completo de una venta de cafeteria desde la mesa.

5. Decisiones de diseño de persistencia

D19. Serializacion Java con tres archivos separados por modulo

Cambio: El analisis solo mencionaba que la informacion debe ser persistente en archivos. En el codigo actual existe un sistema de persistencia con tres interfaces (IPersistenciaCafe, IPersistenciaTiendaDeJuegos, IPersistenciaDataBase) y tres implementaciones de serializacion Java, coordinadas por CentralPersistencia.

Justificacion: La serializacion Java serializa el grafo completo de objetos recursivamente, lo que permite guardar y cargar el estado completo del sistema con pocas lineas. Separar en tres archivos por modulo permite que si solo cambia el cafe, solo se persiste ese archivo. CentralPersistencia actua como fabrica, desacoplando el mecanismo de persistencia del resto del sistema.

D20. Todas las clases del dominio implementan Serializable

Cambio: El analisis no mencionaba Serializable. En el codigo actual Cafe implementa Serializable con serialVersionUID = 1L. Para que la serializacion funcione correctamente, todas las clases referenciadas (Mesa, Producto, Bebida, Pasteleria, Empleado, Turno, etc.) tambien deben implementar Serializable.

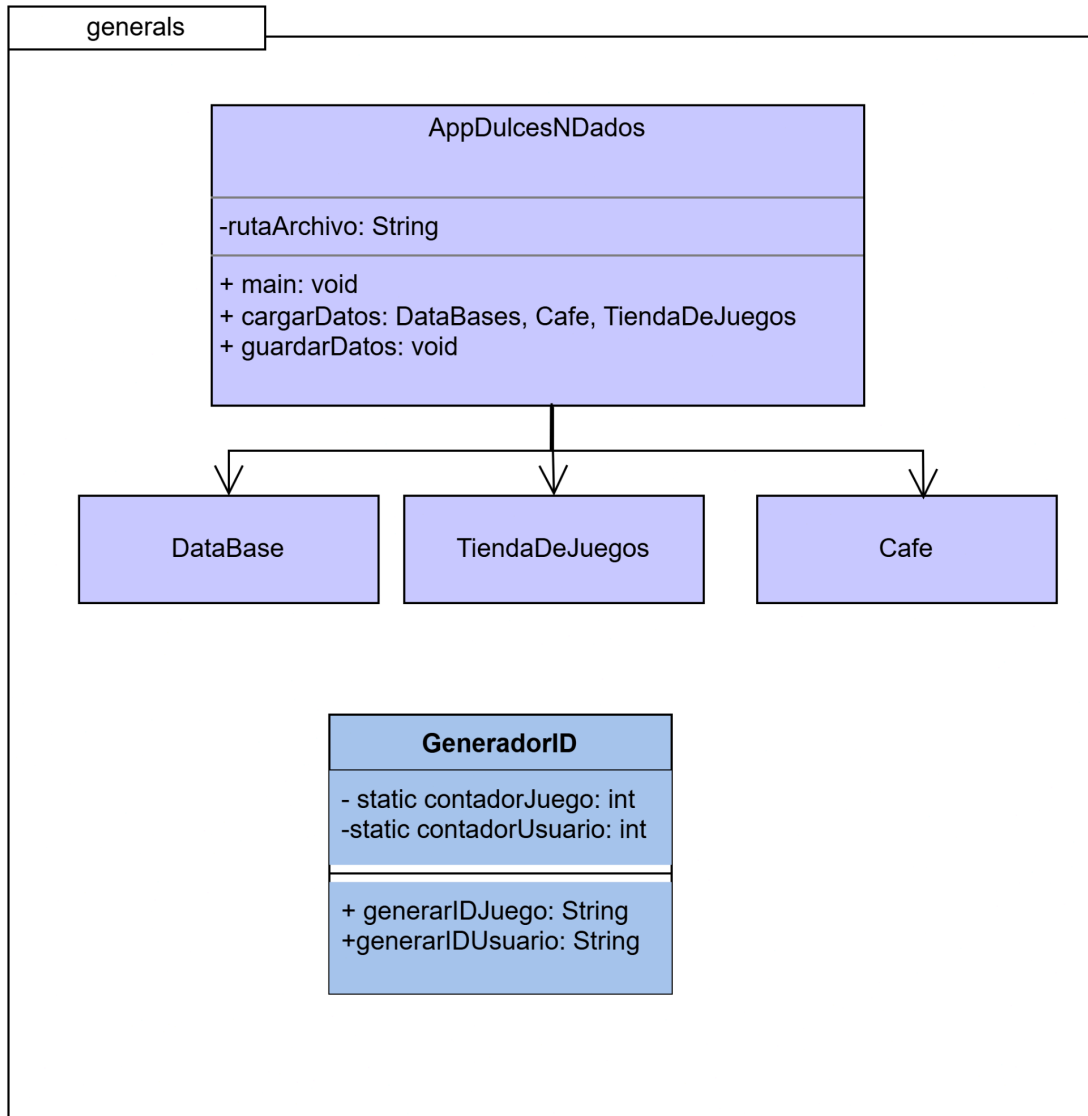
Justificacion: La serializacion Java falla con NotSerializableException si alguna clase en el grafo de objetos no implementa Serializable. El serialVersionUID garantiza compatibilidad entre versiones: si se cambia la clase, se incrementa el numero para que Java detecte archivos guardados con versiones anteriores.

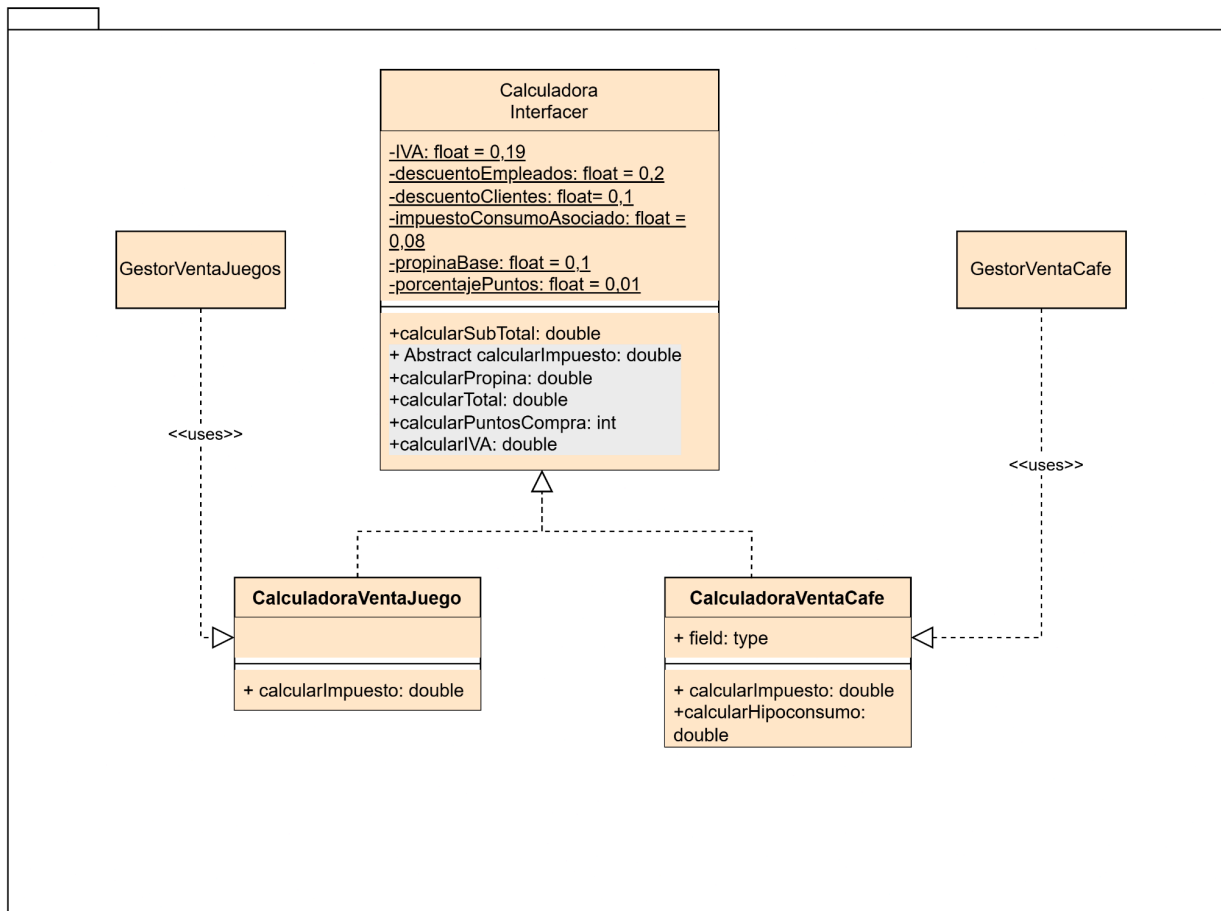
6. Estructura de paquetes

El sistema se organizó en los siguientes paquetes, cada uno con una responsabilidad clara que no estaba explícitamente definida en el análisis original:

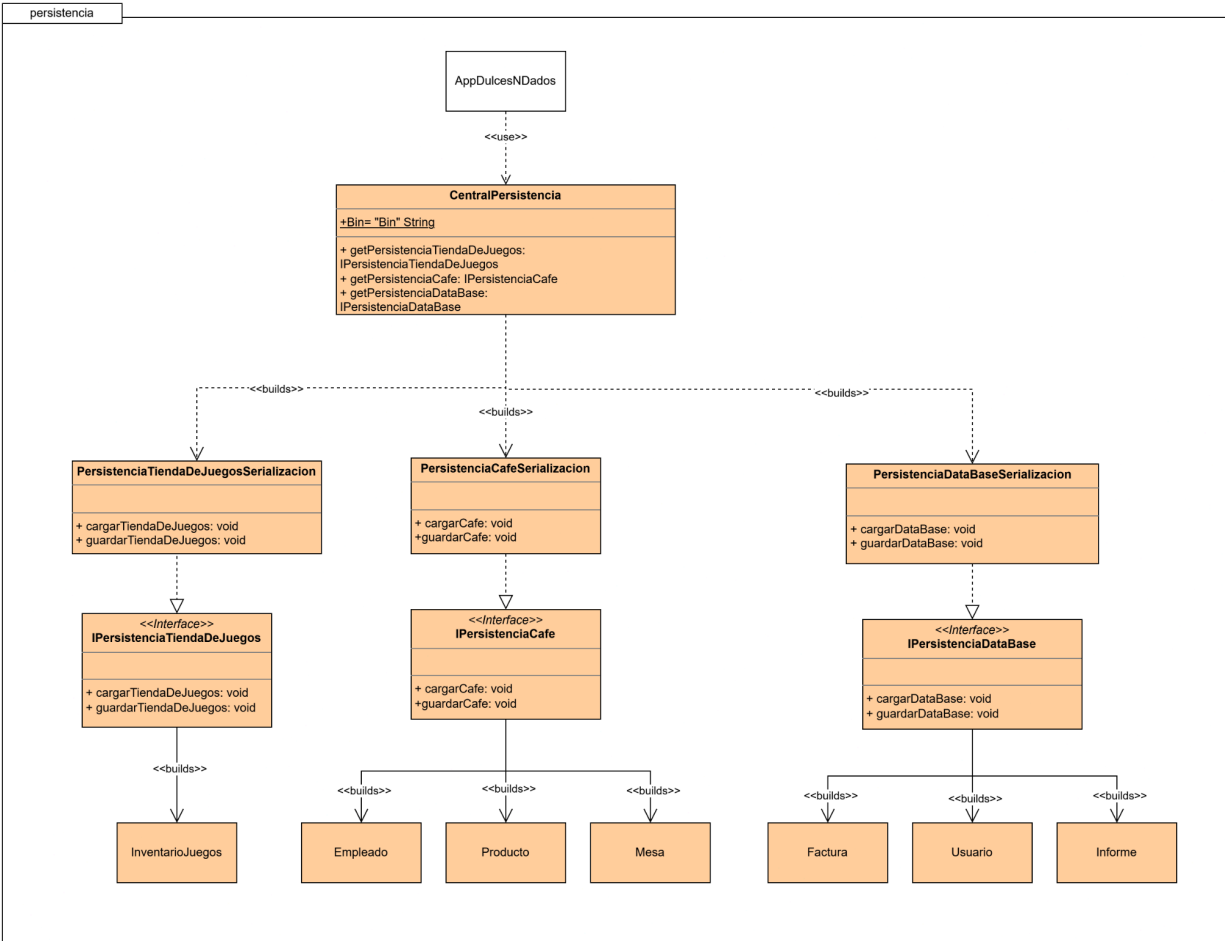
1. cafe — Clases del dominio del cafe: Cafe, Mesa, Producto, Bebida, Pasteleria, GestorVentaCafe.
2. tiendaDeJuegos — Inventario y ventas: TiendaDeJuegos, InventarioJuegos, JuegoDeMesa, JuegoDeMesaFisico, GestorVentaJuegos, GestorInventarioJuegos, Prestamo, TipoDeJuego, TipoInventario, TipoVenta.
3. dataBase — Datos compartidos: DataBase, Usuario, Cliente, Empleado, Mesero, Cocinero, Factura, DetalleVenta, Turno, DiaSemana, Solicitud y sus subclases, Informe.
4. calculadora — Interfaz Calculadora, CalculadoraVentaJuegos, CalculadoraVentaCafe.
5. persistencia — CentralPersistencia, interfaces y clases de serializacion.
6. exceptions — Todas las excepciones personalizadas del sistema.
7. generals — DulcesNDados (punto de entrada), generadorID.

8. Paquete de Generals

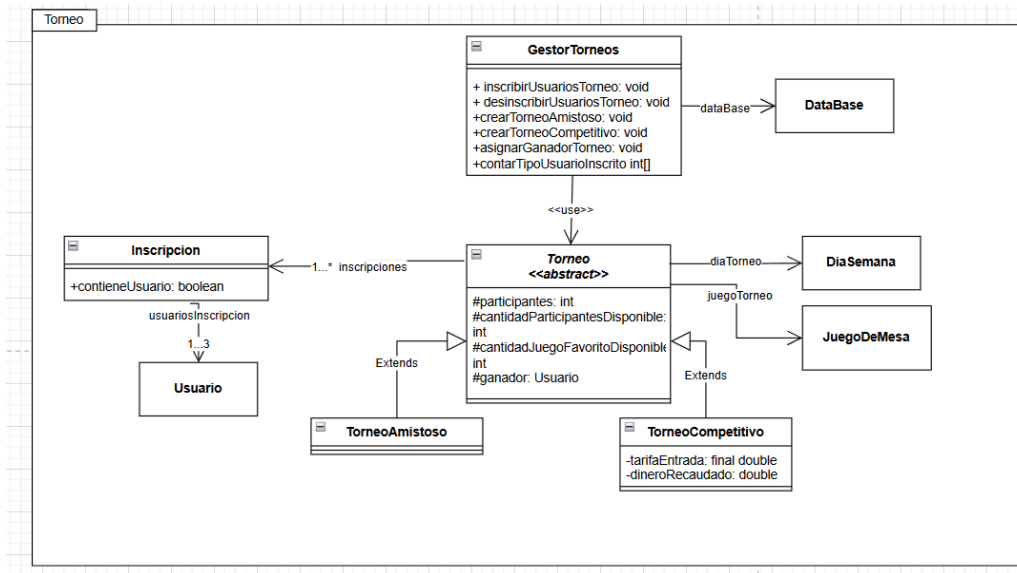




9. Paquete de Calculadora



3. Paquete de Persistencia



5. Paquete de Torneo

Parte de Diseño proyecto 2

7. Decisiones de diseño torneo

A la hora de diseñar el nuevo requerimiento de torneos se tomaron diferentes decisiones de diseño.

Estructura de clases del torneo: Se creó una clase abstracta **Torneo** que contiene los atributos comunes a ambos tipos: la lista de inscripciones, el día de la semana, el juego asociado, los cupos disponibles, y el ganador. De esa clase salen sus hijas **TorneoAmistoso** y **TorneoCompetitivo**. La clase TorneoCompetitivo agrega los atributos precioEntrada y dineroRecaudado, que se actualiza cada vez que se inscriben participantes. Esto permite tratar ambos tipos de torneo de forma uniforme con el GestorTorneos.

Registro de torneos en DataBase: Se decidió que la lista de torneos (ArrayList<Torneo>) quede almacenada directamente en clase DataBase, siguiendo el mismo patrón ya establecido para usuarios, facturas y solicitudes.

Clase Inscripción: Se modeló una clase Inscripción que agrupa entre 1 y 3 usuarios bajo la misma inscripción. El constructor lanza una RestriccionJugadoresException si el tamaño de la lista no cumple con la restricción.

Cupos para fanáticos: El 20% de los cupos se reserva para usuarios que tienen el juego del torneo en una lista de juegos favoritos. Estos cupos se contabilizan y descuentan separadamente de los cupos normales. Si los cupos fanáticos se agotan, el gestor reintenta automáticamente con cupos regulares.

Premio del torneo amistoso: Se decidió utilizar el mecanismo de bonos ya implementado para la entrega anterior, el atributo tieneBonoTorneo en la clase Cliente se activa al asignarle el premio.

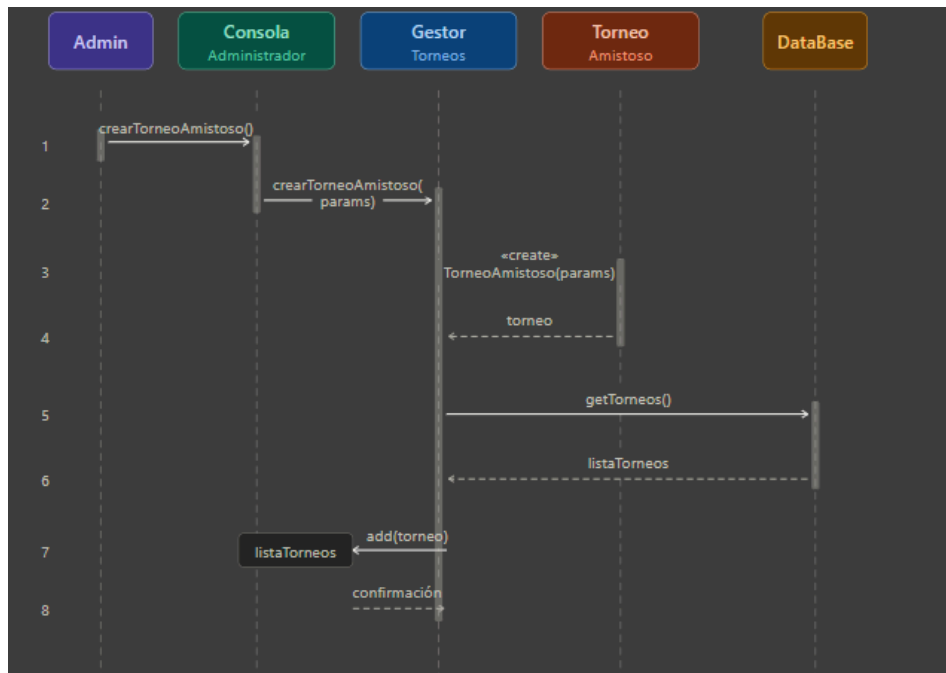
Premio del torneo competitivo: El dinero recaudado se acumula sobre el atributo dineroRecaudado del TorneoCompetitivo con cada inscripción pagada. Al asignar el ganador, el valor se suma a los puntos de fidelidad del cliente ganador.

Movimiento de DiaSemana: Al ser un enum utilizado por el módulo de turnos como por el nuevo módulo de torneos, se decidió mover DiaSemana del paquete de dataBase al paquete generals.

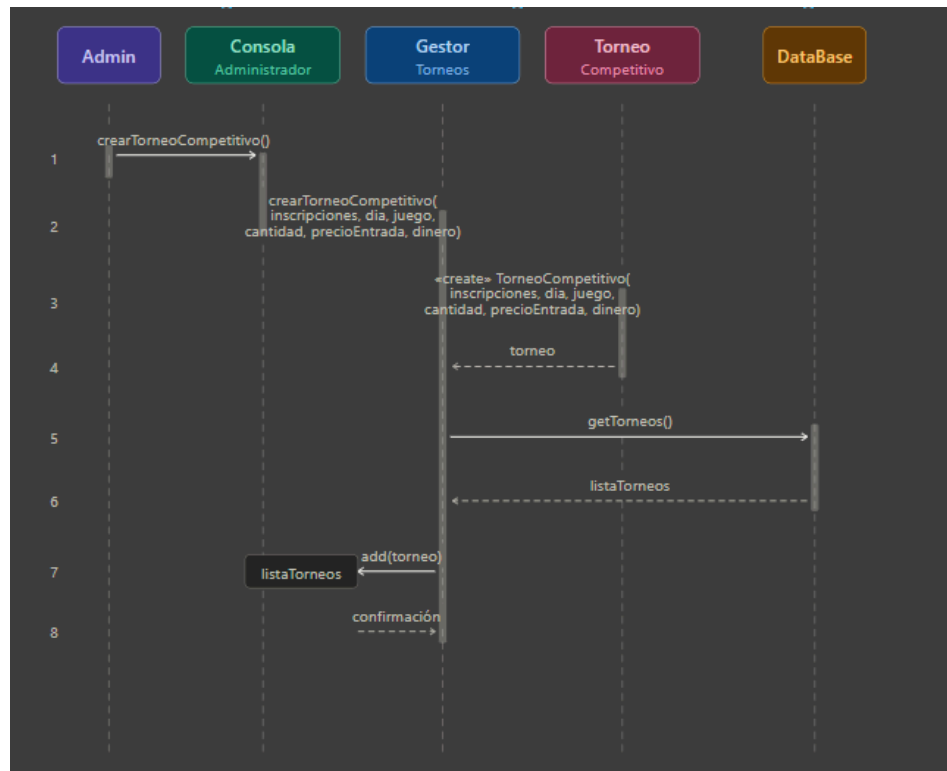
GestorTorneos: Se creó una clase GestorTorneos que recibe la dataBase como dependencia y expone los métodos de creación de torneos, inscripción, desinscripción y asignación de ganador.

Diagramas de flujo:

Diagrama creación torneo amistoso:



Creación torneo Competitivo:



8. Diseño interfaces y consola

Las interfaces de consola se estructuraron en cuatro clases dentro del paquete Consola: MainAdmin, MainCliente, MainEmpleado y ConsolaUtils. Cada una de las tres clases principales tiene un método estático que recibe la instancia central de la aplicación, así la carga y persistencia de datos queda en DulcesNDados y los Main no tienen que preocuparse por eso.

ConsolaUtils es la clase que centraliza toda la lectura y validación de entradas: enteros con rangos, doubles, strings, fechas, etc. La idea es no repetir ese código en cada Main y que cada método de interfaz solo se preocupe por mostrar las opciones y llamar a los gestores.

Las funcionalidades por tipo de usuario son las siguientes:

Administrador: puede registrar empleados, verlos, agregar platillos, ver y probar solicitudes, generar informes, gestionar el inventario de juegos (reparar, reportar robo, mover entre inventarios, reabastecer ventas y préstamo), también puede gestionar horarios, y consultar estado, numero de prestamos y fechas de préstamo de un juego.

Cliente: puede registrarse, iniciar sesión, reservar mesa, solicitar juego prestado, comprar juego, comprar platillo, ver catálogo de juegos, ver el menú del café y consultar sus puntos de fidelidad.

Empleado: puede solicitar cambio o intercambio de turno, sugerir platillos, ver sus turnos y ver sus solicitudes.

9. Pruebas

Las nuevas pruebas del proyecto 2 se organizaron en cuatro clases dentro del paquete `testNuevos`, cada una enfocada en un área distinta.

TestTorneos: prueba la lógica del módulo de los torneos de forma aislada. Probamos el constructor del torneo. También verificamos que `GestorTorneos` agrega los torneos correctamente a la `DataBase` y que se pueden tener dos torneos al mismo tiempo. Para inscripción probamos los casos límite (1, 3 y 4 usuarios y una lista vacía) porque el constructor lanza una excepción con entradas inválidas y ahí es donde se aplica la restricción de máximo 3 participantes. Por último, probamos la inscripción exitosa y los dos casos de error posibles: exceder cupos de fanáticos y exceder cupos regulares.

TestIntegracionHistoriaUsuario: tiene una prueba de integración por cada historia de usuario. Cada prueba simula un flujo completo de principio a fin, con múltiples clases colaborando entre ellas. Cubrimos reserva de mesa, pedido de platillo, solicitud y aprobación de cambio de turno, creación de torneo amistoso, inscripción de fanático, respeto de cupos de fanáticos, aprobación de sugerencia de platillo, descuento en factura para empleado, y el error al mover juegos sin stock. La idea es validar que todo funciona junto, no solo cada clase por separado.

TestInventarioJuegosIntegracion: se enfoca en las operaciones sobre el inventario, que es una de las partes con más estado del sistema. Probamos la validación de disponibilidad en ventas y préstamos, la eliminación del primer juego en ambos inventarios, la adición al historial de préstamos, y el flujo completo de mover un juego de ventas a préstamo verificando que los dos stocks se ajustan bien, además del caso del error cuando no hay stock.

TestSolicitudesTurno: cubre las solicitudes de cambio e intercambio de turno. Verificamos que las solicitudes se crean con los datos correctos, que al aceptar un cambio el empleado realmente se mueve de turno, y que al aceptar un intercambio los dos empleados quedan en el turno del otro. También probamos el caso límite en que se intenta sacar un empleado de turno que ya tiene el mínimo de meseros, lo cual debe lanzar una excepción para no dejar el turno sin personal suficiente.

Parte de diseño Proyecto 3

Esta sección muestra el diseño e implementación del proyecto 3 del sistema de DulcesNDados. El objetivo principal del proyecto fue pasar de una interfaz de usuario basada en consola a una interfaz gráfica funcional usando el framework Swing.

Todas las funcionalidades soportadas en las entregas anteriores siguen siendo utilizables desde la nueva interfaz. Esta parte del documento presenta los cambios al modelo de dominio, las decisiones de diseño de interfaz, los diagramas de clases actualizados y el uso de Inteligencia Artificial Generativa durante el desarrollo.

1. Cambios al modelo de Dominio para el Proyecto 3

Se realizaron cambios mínimos al modelo de dominio para soportar la interfaz gráfica sin afectar la lógica de negocio existente.

1.1 Métodos cargarDatos hechos públicos en DulcesNDados

Cambios: Los métodos cargarDatosDataBase y cargarDatosTienda eran privados en DulcesNDados. En el Proyecto 3 se hicieron públicos.

Justificación: La clase MainWindow del paquete interfaz necesita invocar estos métodos al arrancar la aplicación. Al ser privados, el compilador bloqueaba el acceso desde fuera del paquete generals. El cambio sigue el mismo patrón ya existente de cargarDatosCafe, que ya era público desde el Proyecto 2.

1.2 Paquete interfaz como reemplazo del paquete consola

Cambio: El paquete consola (MainAdmin, MainCliente, MainEmpleado, ConsolaUtils) se mantuvo igual. Se agregó un nuevo paquete interfaz con todas las clases Swing.

Justificación: Mantener el paquete consola permite que DulcesNDados.main() siga funcionando para pruebas. El paquete interfaz es completamente independiente y accede al dominio únicamente a través de la fachada DulcesNDados, respetando el diseño de capas establecido en el Proyecto 2.

1.3 Autenticación diferida en el PanelEmpleado

Cambio: PanelEmpleado recibe null como Empleado desde MainWindow y construye primero un panel de login. PanelContenidoEmpleado y PanelTurnosEmpleado solo se instancian despues de que el empleado se autentica correctamente.

Justificación: PanelTurnosEmpleado invoca empleado.getTurnoLaboral() en su constructor. Si empleado es null en ese momento se lanza NullPointerException. Diferir la construcción del

contenido hasta después de la autenticación garantiza que el objeto Empleado siempre sea válido cuando se usan sus métodos. Este patrón de construcción diferida es coherente con el diseño existente de las solicitudes y gestores del dominio.

2. Decisiones de Diseño de la Interfaz Gráfica Empleado

1. Uso de JPanel como contenedor principal

Se implementó cada vista de usuario (Cliente, Administrador, Empleado) como una clase que extiende JPanel. La ventana principal MainWindow abre una ventana secundaria JFrame por cada rol y coloca el panel correspondiente como su contentPane.

Justificación: Esta decisión permite desacoplar la interfaz de cada rol de la ventana principal de la aplicación. MainWindow puede cargar distintos paneles según el usuario que haga clic en el botón de ingreso, sin modificar la lógica interna de cada interfaz. Además facilita la reutilización y el mantenimiento del código, ya que cada panel es autocontenido.

2. Uso de BorderLayout en PanelEmpleado

Se utilizó un BorderLayout para organizar la interfaz del empleado en dos zonas: el panel de navegación con los botones (WEST) y el área central para mostrar el contenido de cada opción (CENTER).

Justificación: La interfaz del empleado presenta dos zonas claramente diferenciadas con responsabilidades distintas. BorderLayout es el layout más apropiado para este patrón de navegación lateral, ya que garantiza que el sidebar tenga un ancho fijo y el contenido ocupe el espacio restante.

3. Separación entre PanelBotonesEmpleado y PanelContenidoEmpleado

Se crearon dos componentes independientes: PanelBotonesEmpleado, que se encarga únicamente de la navegación, y PanelContenidoEmpleado, que se encarga únicamente de mostrar información.

Justificación: Se aplicó el principio de responsabilidad única. Esta separación reduce el acoplamiento y facilita el mantenimiento y la evolución de la interfaz. PanelEmpleado actúa como coordinador entre ambos: es el único que conoce a los dos componentes y conecta los ActionListeners de los botones con los métodos mostrarPanel() del contenido. Esto evita que PanelBotonesEmpleado conozca directamente a PanelContenidoEmpleado.

4. Uso de CardLayout en PanelContenidoEmpleado

Se utilizó un CardLayout para gestionar las cinco vistas distintas del empleado: Ver Turnos, Ver Solicitudes, Cambio de Horario, Intercambio de Horario y Sugerir Platillo.

Justificación: Utilizar múltiples ventanas habría aumentado la complejidad de navegación y el número de objetos JFrame en memoria. Con CardLayout todas las funcionalidades permanecen dentro de la misma ventana y únicamente cambia el contenido visible. Esto mejora la experiencia de usuario y simplifica la implementación. El mismo patrón se aplicó en PanelCliente para alternar entre la vista de login y la vista principal.

5. Comunicación centralizada desde PanelEmpleado

PanelEmpleado coordina la comunicación entre botones y contenido.

Justificación: Se evitó que PanelBotonesEmpleado conociera directamente a PanelContenidoEmpleado. Esto disminuye el acoplamiento entre componentes y convierte a PanelEmpleado en el controlador de la interfaz.

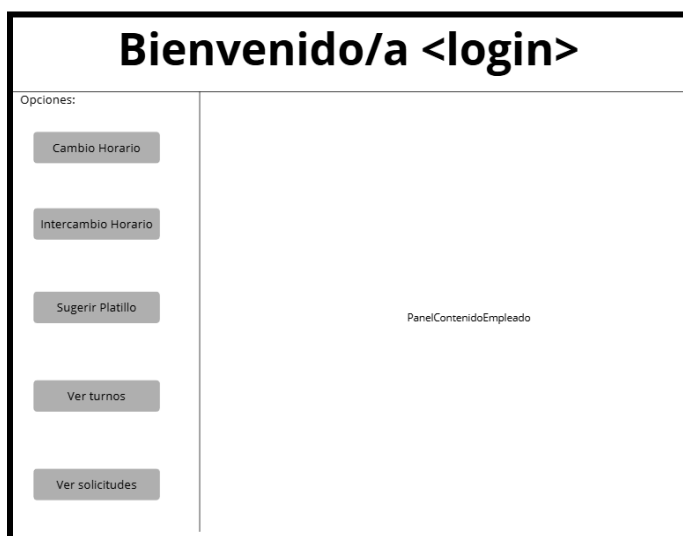
6. Reutilización de la lógica existente

La interfaz gráfica utiliza los métodos ya implementados en las clases de dominio. Operaciones como empleado.solicitarCambioHorario(), tienda.validarDisponibilidadJuegos() o cafe.agregarMesa() ya contienen la lógica de negocio necesaria.

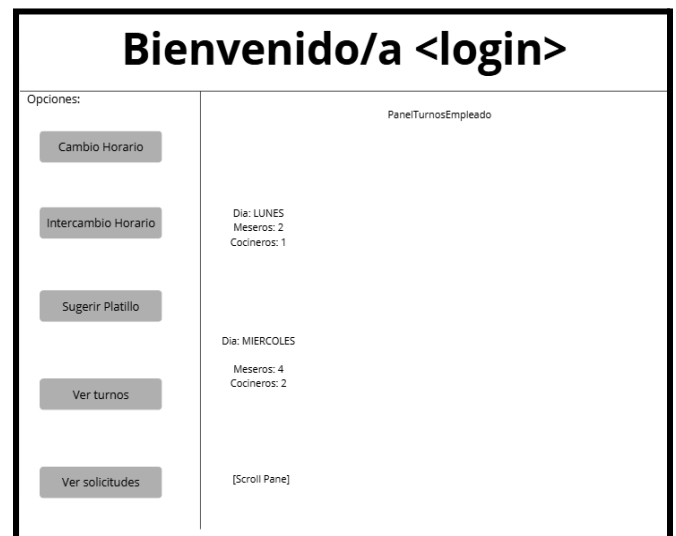
Justificación: La interfaz únicamente captura datos del usuario, los pasa al dominio y presenta los resultados. Esto mantiene separadas las capas de presentación y dominio, evita duplicación de código y garantiza que la lógica de negocio siga siendo la misma independientemente de si se accede desde consola o desde la interfaz gráfica.

1. Interfaz Empleado

Esquema visual ventana principal:



Esquema visual ver turnos:



Esquema visual ver solicitudes:

Bienvenido/a <login>

Opciones:

Cambio Horario

Intercambio Horario

Sugerir Platillo

Ver turnos

Ver solicitudes

PanelSolicitudesEmpleado

SolicitudCambioDeTurno | LUNES -----> MARTES

SolicitudSugerenciaPlatillo | Salchipapa

JList

Esquema visual sugerencia platillo:

Bienvenido/a <login>

Opciones:

Cambio Horario

Intercambio Horario

Sugerir Platillo

Ver turnos

Ver solicitudes

PanelSugerenciaPlatillo

Nombre del platillo: _____

Precio: _____

Enviar Sugerencia

Esquema visual cambio de horario empleado:

Bienvenido/a <login>

Opciones:

Cambio Horario

Intercambio Horario

Sugerir Platillo

Ver turnos

Ver solicitudes

PanelCambioDeHorarioEmpleado

Turno Actual: JComboBox<Turno>

Turno Deseado: JComboBox<Turno>

Enviar Solicitud

Esquema visual intercambio de horario:

Bienvenido/a <login>

Opciones:

Cambio Horario

Intercambio Horario

Sugerir Platillo

Ver turnos

Ver solicitudes

PanelCambioDeHorarioEmpleado

Mi turno: JComboBox<Turno>

Empleado: JComboBox<Empleado>

Turno Deseado: JComboBox<Turno>

Enviar Solicitud

3. Decisiones de Diseño de la Interfaz Gráfica Admin

PanelAdmin implementa las funciones disponibles para el rol Administrador, reutilizando los gestores y entidades del dominio.

1. Uso de JPanel como contenedor principal

Se implementó la vista del administrador mediante una clase PanelAdmin que extiende JPanel.

Justificación: Esta decisión permite desacoplar la interfaz del administrador de la ventana principal. MainWindow puede cargar PanelAdmin como una pestaña más junto a PanelCliente y PanelEmpleado sin modificar la lógica interna de ninguno de los tres paneles, facilitando la reutilización y el mantenimiento del código.

2. Uso de BorderLayout en PanelAdmin

Se utilizó un BorderLayout para organizar la interfaz: un header en la zona norte, la barra lateral de navegación en la zona oeste y el área de contenido en el centro.

Justificación: La interfaz presenta zonas claramente diferenciadas con responsabilidades distintas. BorderLayout es la distribución natural para este esquema de header + sidebar + contenido central.

3. Uso de CardLayout en el área de contenido

Se utilizó un CardLayout para gestionar las 20 vistas del administrador (17 funciones operativas más 3 gráficas).

Justificación: El administrador tiene acceso a un alto número de funcionalidades. Con CardLayout todas permanecen dentro de la misma ventana y únicamente cambia el contenido visible al seleccionar un botón del sidebar, evitando abrir múltiples ventanas y mejorando la experiencia de usuario.

4. Barra lateral envuelta en JScrollPane

La barra lateral izquierda contiene un botón por cada funcionalidad y está envuelta en un JScrollPane con desplazamiento vertical y sin barra horizontal.

Justificación: Los 20 botones del administrador superan el espacio vertical disponible en resoluciones estándar. Envolver el sidebar en un JScrollPane garantiza que todas las opciones sean accesibles sin importar el tamaño de la ventana, evitando que botones queden ocultos o recortados.

5. Panel reutilizable para reabastecimiento

Se implementó un único método privado `crearPanelReabastecer(String titulo, boolean esPrestamo)` que genera tanto el panel de reabastecimiento de ventas como el de préstamo.

Justificación: Ambas funcionalidades comparten exactamente los mismos campos de entrada (nombre, precio, fecha, cantidad, empresa, tipo, jugadores, edad, características, dificultad). Extraer el panel a un método parametrizado elimina duplicación de código y garantiza que cualquier cambio en el formulario se refleje en ambas vistas automáticamente.

6. Reutilización de los gestores del dominio

Cada panel instancia y usa los gestores ya existentes: `GestorInventarioJuegos` para reparar, reportar como robado, mover y reabastecer juegos; `GestorSolicitudesTurno` y `GestorSolicitudesPlatillos` para aprobar solicitudes.

Justificación: La lógica de negocio ya estaba implementada y probada en los gestores del dominio. La interfaz únicamente captura los datos ingresados por el administrador y delega en esos gestores, manteniendo separadas las capas de presentación y dominio sin duplicar código.

7. Tres visualizaciones gráficas con JFreeChart

Se agregaron tres paneles de gráficas dinámicas al `CardLayout`: pastel (distribución de copias de un juego entre venta y préstamo), barras (ventas de cafetería vs juegos en 5 días clasificadas por línea de detalle usando el subtotal sin impuestos) y líneas (número de préstamos por día en la semana ISO a la que pertenece una fecha ingresada por el usuario).

Justificación: `JFreeChart` es compatible con Swing y su componente `ChartPanel` es una subclase de `JPanel`, por lo que cada gráfica se inserta directamente como un card más dentro del `CardLayout` existente sin requerir cambios estructurales. Cada gráfica se regenera dinámicamente al presionar el botón correspondiente, consultando directamente los datos del sistema en ese momento.

4. Decisiones de Diseño de la Interfaz Gráfica Cliente

1. CardLayout para gestión de sesión

Se utilizó un `CardLayout` de dos tarjetas en el nivel raíz de `PanelCliente`: "auth" (formulario de login y registro) y "principal" (interfaz completa del cliente). Ambas tarjetas se construyen en el constructor y la vista de contenido se muestra únicamente después de autenticarse.

Justificación: Mantener ambas vistas en el mismo panel evita destruir y recrear componentes al

cerrar sesión. Si el cliente hace clic en "Cerrar sesión", `cerrarSesion()` pone `clienteActivo = null` y muestra la tarjeta "auth" instantáneamente sin reconstruirla.

2. Uso de BorderLayout en la vista principal

Dentro de la tarjeta "principal" se utilizó un BorderLayout con tres zonas: un header en NORTH con la información de sesión (nombre, mesa activa, puntos y botón cerrar sesión), la barra lateral de navegación en WEST y el área de contenido en CENTER.

Justificación: BorderLayout garantiza que el header tenga altura fija, el sidebar tenga un ancho fijo de 180px (`setPreferredSize()`) y el área de contenido ocupe el espacio restante, adaptándose correctamente al redimensionamiento de la ventana.

3. CardLayout en el área de contenido del cliente

Se utilizó un segundo CardLayout interno para gestionar las siete secciones del cliente: Reservar Mesa, Solicitar Préstamo, Comprar Juego, Comprar Platillo, Catálogo de Juegos, Ver Menú y Mis Puntos de Fidelidad. Los botones del sidebar llaman a `cl.show(contenido, card)` directamente mediante su ActionListener.

Justificación: Con CardLayout todas las secciones permanecen dentro de la misma ventana y únicamente cambia el contenido visible al seleccionar un botón del sidebar, evitando abrir múltiples ventanas. Los botones y sus tarjetas se generan en un único bucle for recorriendo el arreglo `items[][]`, eliminando la repetición de código.

4. Ventana de registro separada del login

El registro de un nuevo cliente se implementó en una ventana independiente creada en el método `mostrarDialogoRegistro()`, separada del formulario de login. La ventana toma usuario, contraseña y juegos favoritos separados por comas, genera el ID con `generadorID.generarIDUsuario()` e inicializa las listas del cliente antes de agregarlo a DataBase.

Justificación: El registro requiere campos adicionales que no son necesarios en el login. Separarlo en una ventana propia mantiene el formulario de login limpio y garantiza que el flujo de registro se complete antes de continuar, ya que la ventana bloquea la aplicación principal hasta que el usuario termine o cancele el proceso.

5. Método actualizarHeader() para retroalimentación en tiempo real

El método actualizarHeader() refresca los tres JLabel del header (nombre del cliente, mesa activa y puntos de fidelidad) cada vez que se produce un cambio de estado relevante: tras el login, tras reservar una mesa, tras comprar un juego y tras ordenar un platillo.

Justificación: El cliente necesita retroalimentación inmediata. Centralizar esta lógica en un único método evita duplicar las tres asignaciones de texto en cada sección y garantiza que el header siempre refleje el estado real del objeto clienteActivo.

6. Métodos de validación reutilizables

Se implementaron dos métodos privados de validación: validarSesion(JLabel), que verifica que clienteActivo != null, y validarMesa(JLabel), que verifica que clienteActivo.getMesa() != null. Ambos muestran el mensaje de error directamente en el JLabel del panel correspondiente y retornan false para cortar el flujo del ActionListener.

Justificación: Estas validaciones se repiten en casi todas las secciones. Entonces lo hacemos métodos reutilizables para no repetir código y además garantiza que el mensaje de error siempre aparezca en el mismo lugar visual dentro de cada sección.

7. Tablas con DefaultTableModel de solo lectura

Las secciones que muestran listas de juegos, platillos o historial de puntos utilizan JTable con DefaultTableModel donde isCellEditable siempre retorna false. Cada tabla tiene un botón de actualización que llama al método de recarga correspondiente (cargarInventario, cargarVenta, cargarMenu, cargarCatalogo).

Justificación: isCellEditable en false evita que el cliente modifique datos del sistema directamente desde la tabla.

8. Reutilización de los gestores del dominio

Cada sección delega sus operaciones en los gestores ya existentes: InventarioJuegos para préstamos y compras de juegos usando validarDisponibilidadJuegos() y eliminarPrimerJuego(), Cafe para reservas de mesa con agregarMesa() y pedidos de platillos con getProductosOrdenados().add().

Justificación: La lógica de negocio ya estaba implementada y probada desde antes, así que la interfaz simplemente toma los datos que ingresa el cliente y se los pasa a los gestores que ya

existían. De esa forma la pantalla solo se encarga de mostrar cosas y recoger datos, sin mezclar eso con la lógica del sistema ni repetir código que ya estaba escrito.

5. Decisiones de Diseño de Gráficas JFreeChart

Se implementaron tres gráficas dinámicas con la librería JFreeChart. Esta librería fue seleccionada por ser compatible con Swing y por ser de uso libre. Cada gráfica se genera con los datos actuales del sistema al momento de abrirla.

Gráfica 1 - Diagrama de Pastel: Distribución de copias de un juego

Muestra la proporción entre copias disponibles para prestar y copias disponibles para vender de un juego seleccionado por el administrador.

- **Fuente de datos:** `inventarioPrestamo.get(nombre).size()` vs `inventarioVenta.get(nombre).size()` en `InventarioJuegos`
- **Tipo:** `ChartFactory.createPieChart3D` con `DefaultPieDataset`

Justificación: El diagrama de pastel 3D hace visualmente inmediata la proporción entre dos categorías complementarias. El administrador puede identificar de un vistazo si un juego está desbalanceado entre sus inventarios de préstamo y venta, lo cual apoya decisiones de reabastecimiento o movimiento de copias.

Gráfica 2 - Diagrama de Barras: Ventas por periodo de 5 días

Muestra las ventas discriminadas entre cafetería y juegos en un rango de 5 días específico, descontando impuestos para mostrar el valor neto.

- **Fuente de datos:** `dataBase.getFacturas()` filtradas por rango de fechas; cada `DetalleVenta` se clasifica según si su `Producto` es `instanceof Bebida` o `Pasteleria` (cafetería) o `instanceof JuegoDeMesa` (juegos)
- **Descuento de impuestos:** $\text{Valor neto} = \text{subtotal} / (1 + \text{tasa})$, usando las tasas de la interfaz Calculadora (IVA 19% para juegos, 8% para cafetería)
- **Tipo:** `ChartFactory.createBarChart` con `DefaultCategoryDataset`, dos series por día (Cafetería y Juegos)

Justificación: Las barras agrupadas permiten comparar fácilmente los dos canales de venta día a día. El administrador puede identificar qué días fueron más productivos y en qué área, apoyando decisiones operativas y de personal.

Gráfica 3 - Diagrama de Líneas: Evolución de reservas en una semana

Muestra el número de reservas de mesa por día de la semana (lunes a domingo) durante una semana seleccionada.

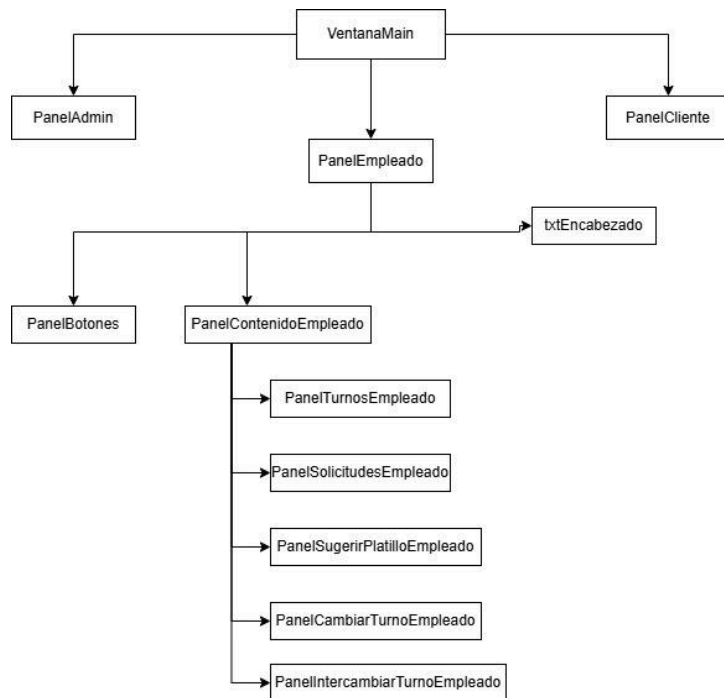
- **Fuente de datos:** `cafe.getMesas()` agrupadas por día usando el campo `fechaReserva` de `Mesa`
- **Tipo:** `ChartFactory.createLineChart` con `DefaultCategoryDataset`

Justificación: El gráfico de líneas es el más adecuado para mostrar tendencias temporales. La evolución día a día de las reservas permite identificar patrones de demanda semanales: días pico y días de baja ocupación. Esta información apoya la planificación de turnos y la asignación de personal.

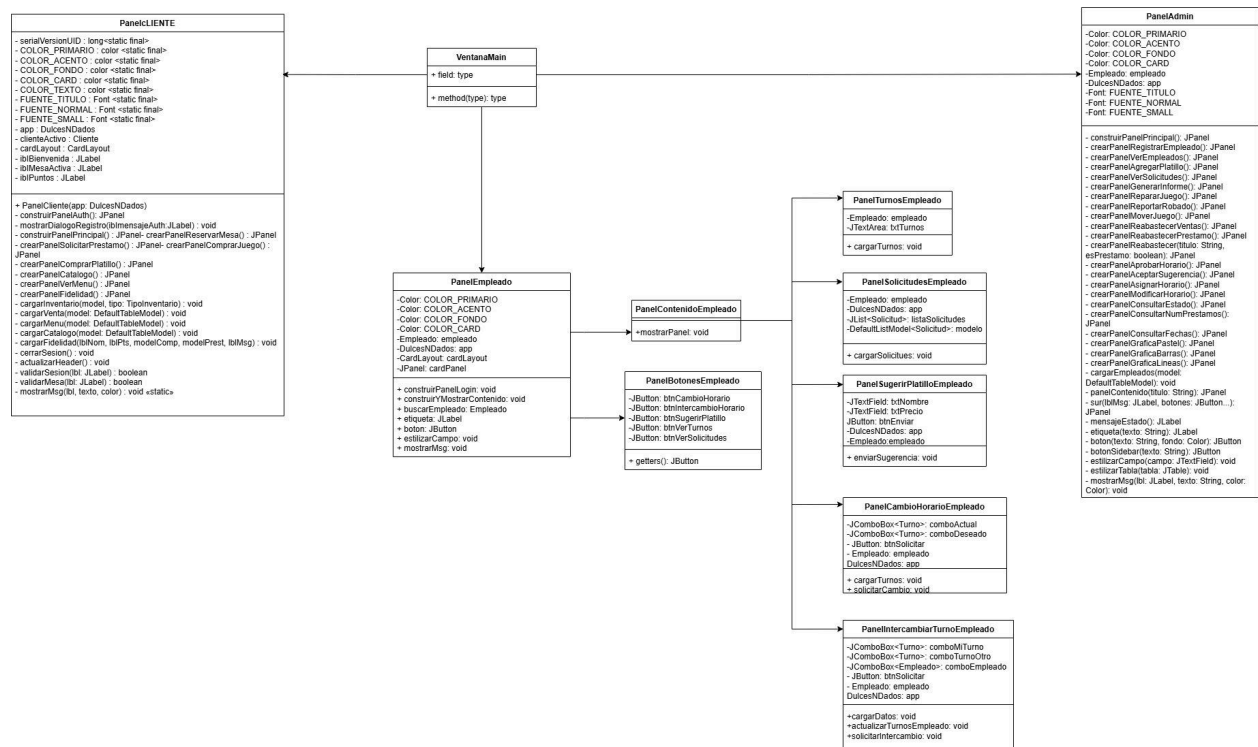
6. Estructura de paquetes del proyecto 3

Se mantiene la estructura de paquetes del Proyecto 2 en su totalidad. Se agrega únicamente el paquete interfaz:

paquete	Contenido y estado
cafe	Cafe, Mesa, Producto, Bebida, Pasteleria, GestorVentaCafe. Sin cambios
tiendaDeJuegos	InventarioJuegos, JuegoDeMesa, JuegoDeMesaFisico, Prestamo, TipoInventario. Sin cambios
dataBase	DataBase, Usuario, Cliente, Empleado, Mesero, Cocinero, Factura, Turno, Solicitudes. Sin cambios
calculadora	Calculadora, CalculadoraVentaJuegos, CalculadoraVentaCafe. Sin cambios
persistencia	CentralPersistencia e interfaces de serializacion. Sin cambios
torneo	GestorTorneos, Torneo, TorneoAmistoso, TorneoCompetitivo, Inscripcion. Sin cambios
generals	DulcesNDados (cargarDatosDataBase y cargarDatosTienda hechos publicos), generadorID, DiaSemana
consola	MainAdmin, MainCliente, MainEmpleado, ConsolaUtils. Conservado sin cambios
interfaz (NUEVO)	MainWindow, PanelCliente, PanelAdmin, PanelEmpleado, PanelBotonesEmpleado, PanelContenidoEmpleado, PanelTurnosEmpleado, PanelGraficas



8. Diagrama de Alto Nivel de Interfaz



8. Diagrama de Alto Nivel de TODO EL PROYECTO

Algo para anotar es que el nuevo paquete, es decir, el paquete de interfaz se identifica con el color rosado. Además, como sé que no es legible, este y los demás diagramas están en el repositorio de github.

